Bikkad IT

Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Session 10: Errors & Exceptions

Practice Questions with Python Code Answers

Compiled by Bikkad IT Institute | Teaching Notes

WHAT IS INSIDE

1. Syntax, Index, Module & Key Errors
2. Type, Value, Name & Attribute Errors
3. try & except Basics
4. Handling Multiple & Specific Errors
5. try - except - else
6. try - except - else - finally
7. raise Exception
8. Creating Custom Exceptions

48 practice questions | every topic: 2 Beginner + 2 Intermediate + 2 Advanced

BEGINNER INTERMEDIATE ADVANCED

OUR LOCATIONS

+91 77220 82225 Pimpri-Chinchwad

+91 77220 82226 Chh. Sambhajinagar

+91 77220 82227 Nashik

+91 77220 82224 Hadapsar, Pune

Session 10: Errors & Exceptions - Practice Solutions

Prepared for: Bikkad IT Institute Students
Topics: Syntax, Index, Module & Key Errors, Type, Value, Name & Attribute Errors, try & except Basics, Handling Multiple & Specific Errors, try - except - else, try - except - else - finally, raise Exception, Creating Custom Exceptions
Levels: 2 Beginner + 2 Intermediate + 2 Advanced questions per topic
Date: October 2026

1. Syntax, Index, Module & Key Errors

Q1. Catch a SyntaxError

BEGINNER

Code written by a student is "print('Hello'" (missing bracket). Use compile() inside try-except to detect the SyntaxError and print its message.

Solution:

```
code = "print('Hello'"

try:
    compile(code, "student.py", "exec")
except SyntaxError as e:
    print("SyntaxError:", e.msg)
```

Output:

```
SyntaxError: '(' was never closed
```

Q2. IndexError in a Queue

BEGINNER

A queue has 3 tokens ["T1", "T2", "T3"]. Try to read index 5 and catch the IndexError.

Solution:

```
queue = ["T1", "T2", "T3"]

try:
    print(queue[5])
except IndexError as e:
    print("IndexError:", e)
```

Output:

```
IndexError: list index out of range
```

Q3. ModuleNotFoundError

INTERMEDIATE

Try to import a module named "pandass" (spelling mistake) and catch ModuleNotFoundError.

Solution:

```
try:
    import pandass
except ModuleNotFoundError as e:
    print("ModuleNotFoundError:", e)
```

Output:

```
ModuleNotFoundError: No module named 'pandass'
```

Q4. KeyError in a Menu

INTERMEDIATE

Menu is {"Tea": 15, "Coffee": 25}. A customer orders "Juice". Catch the KeyError and print "Item not available".

Solution:

```
menu = {"Tea": 15, "Coffee": 25}
order = "Juice"

try:
    print(menu[order])
except KeyError:
    print("Item not available")
```

Output:

```
Item not available
```

Q5. Avoid Errors Before They Happen

ADVANCED

Rewrite both the IndexError and KeyError examples so no exception occurs, using len() check and dict.get().

Solution:

```
queue = ["T1", "T2", "T3"]
index = 5
if index < len(queue):
    print(queue[index])
else:
    print("No token at", index)

menu = {"Tea": 15, "Coffee": 25}
print(menu.get("Juice", "Item not available"))
```

Output:

```
No token at 5
Item not available
```

Q6. Optional Module Import

ADVANCED

Try to import "ujson"; if it is missing, fall back to the standard json module and print which module is used.

Solution:

```
try:
    import ujson as json_lib
except ModuleNotFoundError:
    import json as json_lib

print(json_lib.__name__)
print(json_lib.dumps({"ok": True}))
```

Output:

```
json
{"ok": true}
```

2. Type, Value, Name & Attribute Errors

Q7. TypeError in Billing

BEGINNER

Adding "Total: " + 500 gives a TypeError. Catch it and then fix it using str().

Solution:

```
try:
    print("Total: " + 500)
except TypeError as e:
    print("TypeError:", e)

print("Total: " + str(500))
```

Output:

```
TypeError: can only concatenate str (not "int") to str
Total: 500
```

Q8. ValueError from int()

BEGINNER

A user typed "twelve" for quantity. Convert with int() inside try-except and catch ValueError.

Solution:

```
quantity = "twelve"

try:
    q = int(quantity)
except ValueError as e:
    print("ValueError:", e)
```

Output:

```
ValueError: invalid literal for int() with base 10: 'twelve'
```

Q9. NameError from a Typo

INTERMEDIATE

A variable total_amount is defined but the code prints total_amout. Catch the NameError.

Solution:

```
total_amount = 750

try:
    print(total_amout)
except NameError as e:
    print("NameError:", e)
```

Output:

```
NameError: name 'total_amout' is not defined
```

Q10. AttributeError

INTERMEDIATE

Call append() on a tuple ("Pen", "Book") inside try-except and catch AttributeError.

Solution:

```
items = ("Pen", "Book")

try:
    items.append("Bag")
except AttributeError as e:
    print("AttributeError:", e)
```

Output:

```
AttributeError: 'tuple' object has no attribute 'append'
```

Q11. Identify the Error Type

ADVANCED

Run 4 risky lambdas in a loop: int("x"), [1][3], {}["k"], None.upper(). Catch Exception and print the error class name for each.

Solution:

```
tests = [lambda: int("x"),
         lambda: [1][3],
         lambda: {}["k"],
         lambda: None.upper()]

for t in tests:
    try:
        t()
    except Exception as e:
        print(type(e).__name__)
```

Output:

```
ValueError
IndexError
KeyError
AttributeError
```

Q12. Safe Number Conversion Function

ADVANCED

Write to_number(text) that returns int if possible, else float, else None (catch ValueError). Test with "42", "3.5", "abc".

Solution:

```
def to_number(text):
    try:
        return int(text)
    except ValueError:
        try:
            return float(text)
        except ValueError:
            return None

print(to_number("42"))
print(to_number("3.5"))
print(to_number("abc"))
```

Output:

```
42
3.5
None
```

3. try & except Basics

Q13. Divide Safely

BEGINNER

Divide a bill of 1200 among 0 friends inside try-except and print "Number of friends cannot be zero".

Solution:

```
bill = 1200
friends = 0

try:
    print(bill / friends)
except ZeroDivisionError:
    print("Number of friends cannot be zero")
```

Output:

```
Number of friends cannot be zero
```

Q14. Program Continues After Error

BEGINNER

Show that after handling an error, the rest of the program still runs by printing "Program finished".

Solution:

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Error handled")

print("Program finished")
```

Output:

```
Error handled
Program finished
```

Q15. Get the Error Message

INTERMEDIATE

Catch the exception object with as e and print both its type and message for int("12a").

Solution:

```
try:
    int("12a")
except ValueError as e:
    print(type(e).__name__)
    print(e)
```

Output:

```
ValueError
invalid literal for int() with base 10: '12a'
```

Q16. Ask Again Until Valid

INTERMEDIATE

Keep asking the user for their age until a valid integer is entered. Use a while loop with try-except.

Solution:

```
while True:
    text = input("Enter age: ")
    try:
        age = int(text)
        break
    except ValueError:
        print("Please enter a number")

print("Age is", age)
```

Output (with sample input):

```
Enter age: abc
Please enter a number
Enter age: 2x
Please enter a number
Enter age: 21
Age is 21
```

Q17. Why Handle Exceptions

ADVANCED

Process a list of prices ["100", "abc", "250", "", "75"]. Without handling, one bad value stops everything. With try-except, skip bad values and total the good ones.

Solution:

```
prices = ["100", "abc", "250", "", "75"]
total = 0
bad = 0

for p in prices:
    try:
        total += int(p)
    except ValueError:
        bad += 1

print(total)
print(bad)
```

Output:

```
425
2
```

Q18. Exception Hierarchy

ADVANCED

Show that ZeroDivisionError is a subclass of ArithmeticError and Exception using issubclass(), and catch it using except ArithmeticError.

Solution:

```
print(issubclass(ZeroDivisionError, ArithmeticError))
print(issubclass(ArithmeticError, Exception))

try:
    5 / 0
except ArithmeticError as e:
    print("Caught by parent class:", type(e).__name__)
```

Output:

```
True
True
Caught by parent class: ZeroDivisionError
```

4. Handling Multiple & Specific Errors

Q19. Two except Blocks

BEGINNER

Convert "abc" to int and divide by it. Write separate except blocks for ValueError and ZeroDivisionError.

Solution:

```
value = "abc"

try:
    result = 100 / int(value)
except ValueError:
    print("Not a number")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Output:

```
Not a number
```

Q20. One except for Many Errors

BEGINNER

Catch ValueError and ZeroDivisionError in a single except using a tuple, for the value "0".

Solution:

```
value = "0"

try:
    result = 100 / int(value)
except (ValueError, ZeroDivisionError) as e:
    print("Invalid input:", type(e).__name__)
```

Output:

```
Invalid input: ZeroDivisionError
```

Q21. Calculator with Specific Errors

INTERMEDIATE

Write calc(a, op, b) that handles ZeroDivisionError, KeyError for unknown operators and TypeError for wrong types.

Solution:

```
def calc(a, op, b):
    try:
        operations = {"+": lambda: a + b, "-": lambda: a - b,
                      "*": lambda: a * b, "/": lambda: a / b}
        return operations[op]()
    except ZeroDivisionError:
        return "Divide by zero"
    except KeyError:
        return "Unknown operator"
    except TypeError:
        return "Wrong data type"

print(calc(10, "/", 0))
print(calc(10, "%", 3))
print(calc(10, "+", "5"))
print(calc(10, "*", 3))
```

Output:

```
Divide by zero
Unknown operator
Wrong data type
30
```

Q22. Order of except Blocks

INTERMEDIATE

Put except Exception first and except ValueError second. Show that the specific block never runs, then fix the order.

Solution:

```
try:
    int("x")
except Exception:
    print("General handler ran")
except ValueError:
    print("Specific handler ran")

try:
    int("x")
except ValueError:
    print("Specific handler ran")
except Exception:
    print("General handler ran")
```

Output:

```
General handler ran
Specific handler ran
```

Q23. Read Values from a Record

ADVANCED

A record is {"name": "Om", "marks": [80, "NA", 90]}. Calculate the average while handling KeyError (missing marks), TypeError (non-number) separately.

Solution:

```
record = {"name": "Om", "marks": [80, "NA", 90]}

try:
    marks = record["marks"]
    print(sum(marks) / len(marks))
except KeyError:
    print("Marks missing")
except TypeError:
    clean = [m for m in record["marks"] if isinstance(m, int)]
    print("Skipped invalid marks")
    print(sum(clean) / len(clean))
```

Output:

```
Skipped invalid marks
85.0
```

Q24. Generic Handler as Last Resort

ADVANCED

Run a list of risky tasks. Handle ValueError and IndexError specifically and catch everything else with a final except Exception.

Solution:

```
tasks = [lambda: int("x"),
         lambda: [][0],
         lambda: open("/no/such/file.txt")]

for task in tasks:
    try:
        task()
    except ValueError:
        print("Bad value")
    except IndexError:
        print("Bad index")
    except Exception as e:
        print("Other error:", type(e).__name__)
```

Output:

```
Bad value
Bad index
Other error: FileNotFoundError
```

5. try - except - else

Q25. else Runs Only When No Error

BEGINNER

Convert "45" to int. If successful, the else block should print "Converted successfully".

Solution:

```
try:
    n = int("45")
except ValueError:
    print("Conversion failed")
else:
    print("Converted successfully", n)
```

Output:

```
Converted successfully 45
```

Q26. else Skipped on Error

BEGINNER

Repeat the same with "4x5" to show the else block is skipped.

Solution:

```
try:
    n = int("4x5")
except ValueError:
    print("Conversion failed")
else:
    print("Converted successfully", n)
```

Output:

```
Conversion failed
```

Q27. Login Example

INTERMEDIATE

Look up a username in users = {"om": "1234"}. If found (else block), check the password.

Solution:

```
users = {"om": "1234"}
username = "om"
password = "1234"

try:
    stored = users[username]
except KeyError:
    print("User not found")
else:
    if stored == password:
        print("Login successful")
    else:
        print("Wrong password")
```

Output:

```
Login successful
```

Q28. Keep try Small

INTERMEDIATE

Write a percentage calculator where only the risky division is in try, and printing the result is in else.

Solution:

```
obtained = 450
total = 500

try:
    percent = obtained / total * 100
except ZeroDivisionError:
    print("Total cannot be zero")
else:
    print(percent)
```

Output:

```
90.0
```

Q29. Process Many Files Safely

ADVANCED

For a list of numbers as text ["10", "0", "x", "5"], compute 100/n. Print errors in except and successful results in else.

Solution:

```
values = ["10", "0", "x", "5"]

for v in values:
    try:
        result = 100 / int(v)
    except (ValueError, ZeroDivisionError) as e:
        print(v, "error:", type(e).__name__)
    else:
        print(v, "result:", result)
```

Output:

```
10 result: 10.0
0 error: ZeroDivisionError
x error: ValueError
5 result: 20.0
```

Q30. Order Placement

ADVANCED

Stock is {"Pen": 5}. place_order(item, qty) raises errors for missing item (KeyError) and insufficient stock (ValueError). Use else to reduce stock only on success.

Solution:

```
stock = {"Pen": 5}

def place_order(item, qty):
    try:
        available = stock[item]
        if qty > available:
            raise ValueError("Insufficient stock")
    except KeyError:
        print("No such item:", item)
    except ValueError as e:
        print(e)
    else:
        stock[item] -= qty
        print("Order placed. Left:", stock[item])

place_order("Pen", 3)
place_order("Pen", 4)
place_order("Ink", 1)
```

Output:

```
Order placed. Left: 2
Insufficient stock
No such item: Ink
```

6. try - except - else - finally

Q31. finally Always Runs

BEGINNER

Show finally runs both when there is no error and when there is an error.

Solution:

```
for value in ["8", "x"]:
    try:
        print(int(value))
    except ValueError:
        print("Error")
    finally:
        print("Done with", value)
```

Output:

```
8
Done with 8
Error
Done with x
```

Q32. Closing a File in finally

BEGINNER

Open a file "notes.txt" for writing, write a line, and close it in finally.

Solution:

```
f = open("notes.txt", "w")
try:
    f.write("Bikkad IT notes")
finally:
    f.close()
    print("File closed:", f.closed)
```

Output:

```
File closed: True
```

Q33. All Four Blocks

INTERMEDIATE

Write a bank withdrawal of 2000 from balance 5000 using try, except, else and finally, printing a message in each.

Solution:

```
balance = 5000
amount = 2000

try:
    if amount > balance:
        raise ValueError("Insufficient balance")
except ValueError as e:
    print(e)
else:
    balance -= amount
    print("Withdrawn. Balance:", balance)
finally:
    print("Thank you for banking with us")
```

Output:

```
Withdrawn. Balance: 3000
Thank you for banking with us
```

Q34. finally with return

INTERMEDIATE

Write a function that returns from try. Show that finally still runs before the function returns.

Solution:

```
def get_data():
    try:
        return "data"
    finally:
        print("Cleaning up")

print(get_data())
```

Output:

```
Cleaning up
data
```

Q35. Database Connection Simulation

ADVANCED

Simulate connect -> query -> close. The query fails with a ZeroDivisionError. Ensure close always happens.

Solution:

```
def run_query():
    print("Connected")
    try:
        result = 10 / 0
    except ZeroDivisionError:
        print("Query failed")
    else:
        print("Query result", result)
    finally:
        print("Connection closed")

run_query()
```

Output:

```
Connected
Query failed
Connection closed
```

Q36. Timing an Operation

ADVANCED

Measure how long a task takes using time.time() in try and print the duration in finally, even if an error happens.

Solution:

```
import time

start = time.time()
try:
    total = sum(range(1000000))
    print(total)
finally:
    duration = time.time() - start
    print("Took less than 1 second:", duration < 1)
```

Output:

```
499999500000
Took less than 1 second: True
```

7. raise Exception

Q37. Raise for Negative Age

BEGINNER

Write set_age(age) that raises ValueError("Age cannot be negative") for negative values. Call it with -5 inside try-except.

Solution:

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

try:
    set_age(-5)
except ValueError as e:
    print(e)
```

Output:

```
Age cannot be negative
```

Q38. Raise for Empty Name

BEGINNER

Write register(name) that raises ValueError if the name is empty, otherwise prints "Registered".

Solution:

```
def register(name):
    if name.strip() == "":
        raise ValueError("Name is required")
    print("Registered", name)

register("Kavya")
try:
    register("   ")
except ValueError as e:
    print(e)
```

Output:

```
Registered Kavya
Name is required
```

Q39. Raise TypeError for Wrong Type

INTERMEDIATE

Write area(side) that raises TypeError if side is not a number.

Solution:

```
def area(side):
    if not isinstance(side, (int, float)):
        raise TypeError("Side must be a number")
    return side * side

print(area(4))
try:
    area("4")
except TypeError as e:
    print(e)
```

Output:

```
16
Side must be a number
```

Q40. Withdraw with raise

INTERMEDIATE

Write withdraw(balance, amount) that raises ValueError when amount > balance or amount <= 0.

Solution:

```
def withdraw(balance, amount):
    if amount <= 0:
        raise ValueError("Amount must be positive")
    if amount > balance:
        raise ValueError("Insufficient balance")
    return balance - amount

for amt in [500, -10, 9000]:
    try:
        print(withdraw(5000, amt))
    except ValueError as e:
        print(e)
```

Output:

```
4500
Amount must be positive
Insufficient balance
```

Q41. Re-raise After Logging

ADVANCED

Inside except, print a log message and then re-raise the same error with raise. Catch it again outside.

Solution:

```
def process(value):
    try:
        return int(value)
    except ValueError:
        print("LOG: bad value received:", value)
        raise

try:
    process("abc")
except ValueError as e:
    print("Handled outside:", e)
```

Output:

```
LOG: bad value received: abc
Handled outside: invalid literal for int() with base 10: 'abc'
```

Q42. Exception Chaining with from

ADVANCED

Convert a KeyError from a config lookup into a RuntimeError using raise ... from e. Print both the new error and its cause.

Solution:

```
config = {"host": "localhost"}

def get_port():
    try:
        return config["port"]
    except KeyError as e:
        raise RuntimeError("Port not configured") from e

try:
    get_port()
except RuntimeError as e:
    print(e)
    print(repr(e.__cause__))
```

Output:

```
Port not configured
KeyError('port')
```

8. Creating Custom Exceptions

Q43. Simple Custom Exception

BEGINNER

Create class InsufficientBalanceError(Exception). Raise it and catch it.

Solution:

```
class InsufficientBalanceError(Exception):
    pass

try:
    raise InsufficientBalanceError("Balance too low")
except InsufficientBalanceError as e:
    print(e)
```

Output:

```
Balance too low
```

Q44. Use in a Function

BEGINNER

Create InvalidMarksError and raise it in check_marks(m) if m is not between 0 and 100.

Solution:

```
class InvalidMarksError(Exception):
    pass

def check_marks(m):
    if m < 0 or m > 100:
        raise InvalidMarksError(f"{m} is not valid marks")
    return m

for m in [85, 120]:
    try:
        print(check_marks(m))
    except InvalidMarksError as e:
        print(e)
```

Output:

```
85
120 is not valid marks
```

Q45. Custom Exception with Extra Data

INTERMEDIATE

Create OutOfStockError(item, requested, available) that stores these values and builds a message. Print the stored values when caught.

Solution:

```
class OutOfStockError(Exception):
    def __init__(self, item, requested, available):
        self.item = item
        self.requested = requested
        self.available = available
        msg = f"{item}: asked {requested}, only {available} left"
        super().__init__(msg)

try:
    raise OutOfStockError("Pen", 10, 4)
except OutOfStockError as e:
    print(e)
    print(e.requested - e.available)
```

Output:

```
Pen: asked 10, only 4 left
6
```

Q46. Custom Exception Hierarchy

INTERMEDIATE

Create BankError(Exception) with children LowBalanceError and AccountLockedError. Catch both using except BankError.

Solution:

```
class BankError(Exception):
    pass

class LowBalanceError(BankError):
    pass

class AccountLockedError(BankError):
    pass

errors = [LowBalanceError("Low balance"),
          AccountLockedError("Account locked")]
for err in errors:
    try:
        raise err
    except BankError as e:
        print(type(e).__name__, "-", e)
```

Output:

```
LowBalanceError - Low balance
AccountLockedError - Account locked
```

Q47. Bank Account with Custom Errors

ADVANCED

Create class Account with withdraw() that raises LowBalanceError, and AccountLockedError after 3 wrong PIN attempts.

Solution:

```
class LowBalanceError(Exception):
    pass

class AccountLockedError(Exception):
    pass

class Account:
    def __init__(self, pin, balance):
        self.pin = pin
        self.balance = balance
        self.wrong = 0

    def withdraw(self, pin, amount):
        if self.wrong >= 3:
            raise AccountLockedError("Account locked")
        if pin != self.pin:
            self.wrong += 1
            return "Wrong PIN"
        if amount > self.balance:
            raise LowBalanceError("Not enough balance")
        self.balance -= amount
        return "Success"

acc = Account(1234, 3000)
try:
    print(acc.withdraw(1234, 5000))
except LowBalanceError as e:
    print(e)

for i in range(4):
    try:
        print(acc.withdraw(1111, 100))
    except AccountLockedError as e:
        print(e)
```

Output:

```
Not enough balance
Wrong PIN
Wrong PIN
Wrong PIN
Account locked
```

Q48. Validate a Form

ADVANCED

Create ValidationError(field, message). Write validate(form) that raises it for missing email or short password. Collect all errors in a list instead of stopping at the first.

Solution:

```
class ValidationError(Exception):
    def __init__(self, field, message):
        self.field = field
        super().__init__(f"{field}: {message}")

def validate(form):
    errors = []
    email = form.get("email", "")
    password = form.get("password", "")
    checks = [
        ("email", lambda: "@" in email, "invalid email"),
        ("password", lambda: len(password) >= 8, "too short"),
    ]
    for field, rule, msg in checks:
        try:
            if not rule():
                raise ValidationError(field, msg)
        except ValidationError as e:
            errors.append(str(e))
    return errors

print(validate({"email": "om.mail.com", "password": "123"}))
print(validate({"email": "om@mail.com", "password": "secret123"}))
```

Output:

```
['email: invalid email', 'password: too short']
[]
```

End of Solutions

Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: October 2026